

ASSIGNMENT:Sentiment Analysis using Transformer Models – Hugging Face

STEP 1 — Environment Setup

```
!pip install -U transformers datasets evaluate accelerate
```

Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)

Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)

Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)

Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate)

Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate)

Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate)

Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)

Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0)

Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3)

Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2.8.2)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets) (2025.3)

Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (8.3.1)

Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (1.5.4)

Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (13.9.4)

Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->transformers) (0.0.2)

Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!)

Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!)

Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!)

Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!)

Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,!)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas->datasets)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=2.0.0)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=2.0.0->accelerate)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0)

Downloading transformers-5.3.0-py3-none-any.whl (10.7 MB)

10.7/10.7 MB 33.2 MB/s eta 0:00:00

Downloading datasets-4.7.0-py3-none-any.whl (527 kB)

527.5/527.5 kB 23.2 MB/s eta 0:00:00

Downloading evaluate-0.4.6-py3-none-any.whl (84 kB)

84.1/84.1 kB 2.9 MB/s eta 0:00:00

Downloading accelerate-1.13.0-py3-none-any.whl (383 kB)

383.7/383.7 kB 12.0 MB/s eta 0:00:00

Downloading pyarrow-23.0.1-cp312-cp312-manylinux_2_28_x86_64.whl (47.6 MB)

47.6/47.6 MB 17.6 MB/s eta 0:00:00

Installing collected packages: pyarrow, datasets, accelerate, transformers, evaluate

Attempting uninstall: pyarrow

Found existing installation: pyarrow 18.1.0

Uninstalling pyarrow-18.1.0:

Successfully uninstalled pyarrow-18.1.0

Attempting uninstall: datasets

Found existing installation: datasets 4.0.0

Uninstalling datasets-4.0.0:

Successfully uninstalled datasets-4.0.0

Attempting uninstall: accelerate

Found existing installation: accelerate 1.12.0

Uninstalling accelerate-1.12.0:

Successfully uninstalled accelerate-1.12.0

Attempting uninstall: transformers

Found existing installation: transformers 5.0.0

Uninstalling transformers-5.0.0:

Successfully uninstalled transformers-5.0.0

Successfully installed accelerate-1.13.0 datasets-4.7.0 evaluate-0.4.6 pyarrow-23.0.1 transformers-5.3.0

Environment setup installs required libraries such as Transformers, Datasets, and Accelerate to enable training of Transformer models using Hugging Face.

STEP 2 — Import Libraries

```
import numpy as np
import pandas as pd
import torch

from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    Trainer,
    TrainingArguments
)
```

```
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

These libraries help with dataset loading, tokenization, model building, training, and evaluation of the sentiment classification model.

STEP 3 — Load Dataset

```
dataset = load_dataset("imdb")
dataset
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:      7.81k/? [00:00<00:00, 229kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
plain_text/train-00000-of-00001.parquet: 100%                21.0M/21.0M [00:01<00:00, 173MB/s]
plain_text/test-00000-of-00001.parquet: 100%                20.5M/20.5M [00:01<00:00, 16.0MB/s]
plain_text/unsupervised-00000-of-00001.p(...): 100%         42.0M/42.0M [00:00<00:00, 203MB/s]
Generating train split: 100%                                25000/25000 [00:00<00:00, 39068.23 examples/s]
Generating test split: 100%                                25000/25000 [00:00<00:00, 48718.05 examples/s]
Generating unsupervised split: 100%                        50000/50000 [00:00<00:00, 82171.82 examples/s]
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

```
import torch
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
```

```
True
Tesla T4
```

The IMDB Movie Reviews dataset contains 50,000 movie reviews labeled as positive or negative sentiment. It is commonly used for binary sentiment classification tasks in NLP. The dataset is balanced with equal numbers of positive and negative reviews. The training set contains 25,000 reviews and the test set contains 25,000 reviews. Each review is associated with a sentiment label

STEP 4 — Tokenization (Text Preprocessing)

```
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")

def tokenize_function(example):
    return tokenizer(
        example["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

tokenized_dataset = dataset.map(tokenize_function, batched=True)
```

```

config.json: 100%                                483/483 [00:00<00:00, 21.8kB/s]
tokenizer_config.json: 100%                        48.0/48.0 [00:00<00:00, 1.67kB/s]
vocab.txt:      232k/? [00:00<00:00, 882kB/s]
tokenizer.json:  466k/? [00:00<00:00, 1.46MB/s]
Map: 100%                                25000/25000 [00:26<00:00, 953.65 examples/s]
Map: 100%                                25000/25000 [00:27<00:00, 768.91 examples/s]
Map: 100%                                50000/50000 [00:50<00:00, 1020.00 examples/s]

```

Tokenization converts text into numerical tokens that the Transformer model can process. Padding ensures equal sequence length and truncation prevents sequences from exceeding the maximum length.

STEP 5 — Prepare Dataset for Training

```

tokenized_dataset = tokenized_dataset.rename_column("label", "labels")

tokenized_dataset.set_format(
    "torch",
    columns=["input_ids", "attention_mask", "labels"]
)

train_dataset = tokenized_dataset["train"]
test_dataset = tokenized_dataset["test"]

```

The dataset is converted into PyTorch tensor format so that it can be used directly by the Transformer model during training.

STEP 6 — Load Pretrained Transformer Model

```

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2
)

```

```

model.safetensors: 92%                                247M/268M [00:02<00:00, 255MB/s]
Loading weights: 100%                                100/100 [00:00<00:00, 3236.80it/s]

```

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased

Key	Status
-----	-----
vocab_projector.bias	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
vocab_layer_norm.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
classifier.weight	MISSING
classifier.bias	MISSING
pre_classifier.bias	MISSING
pre_classifier.weight	MISSING

Notes:

- **UNEXPECTED** :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- **MISSING** :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task

A pretrained DistilBERT model is loaded and fine-tuned for sentiment classification with two output classes: positive and negative.

STEP 7 — Training Configuration

```

training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=1,
    weight_decay=0.01,
    logging_steps=100
)

```

Training arguments define hyperparameters such as learning rate, batch size, and number of epochs used during model training.

STEP 8 — Define Evaluation Metrics

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average="binary"
    )

    acc = accuracy_score(labels, predictions)

    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

STEP 9 — Train Model

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)

trainer.train()
```

 [1563/1563 05:26, Epoch 1/1]

Step	Training Loss
100	0.570735
200	0.382419
300	0.378473
400	0.349527
500	0.339601
600	0.357175
700	0.346815
800	0.347708
900	0.310524
1000	0.324955
1100	0.310634
1200	0.327316
1300	0.297809
1400	0.304840
1500	0.305725

Writing model shards: 100%

1/1 [00:00<00:00, 1.25it/s]

Writing model shards: 100%

1/1 [00:06<00:00, 6.45s/it]

Writing model shards: 100%

1/1 [00:06<00:00, 6.42s/it]

Writing model shards: 100%

1/1 [00:01<00:00, 1.58s/it]

TrainOutput(global_step=1563, training_loss=0.34810126384556944, metrics={'train_runtime': 328.1921, 'train_samples_per_second': 76.175, 'train_steps_per_second': 4.762, 'total_flos': 827921241600000.0, 'train_loss':

You will see training loss decreasing.

STEP 10 — Model Evaluation

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
```

```
compute_metrics=compute_metrics
)
```

```
import numpy as np
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

predictions = trainer.predict(test_dataset)

y_pred = np.argmax(predictions.predictions, axis=1)
y_true = predictions.label_ids

precision, recall, f1, _ = precision_recall_fscore_support(y_true, y_pred, average='binary')
accuracy = accuracy_score(y_true, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

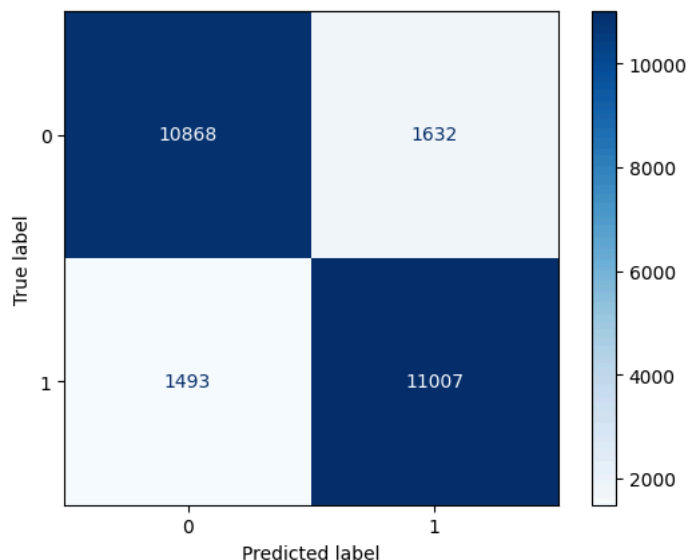
```
Accuracy: 0.875
Precision: 0.8708758604319962
Recall: 0.88056
F1 Score: 0.8756911571661562
```

STEP 11 — Confusion Matrix

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

predictions = trainer.predict(test_dataset)
y_pred = np.argmax(predictions.predictions, axis=1)
y_true = predictions.label_ids

cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues")
plt.show()
```



```
text = "The movie was absolutely fantastic with brilliant acting."

inputs = tokenizer(text, return_tensors="pt", truncation=True, padding=True)

# Move inputs to GPU
inputs = {key: value.to("cuda") for key, value in inputs.items()}

# Model prediction
outputs = model(**inputs)

prediction = torch.argmax(outputs.logits, dim=1).item()

if prediction == 1:
    print("Positive Review 😊")
else:
    print("Negative Review 😞")
```

Positive Review 😊

